

**Perancangan dan Pengembangan EduLab: Sistem Manajemen Tugas
& Penilaian Praktikum**



OLEH

Muh Yusuf

NIM: E1E124044

JURUSAN TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS HALU OLEO

2026

DAFTAR ISI

BAB I	PENDAHULUAN	1
1.1	Latar Belakang	1
1.2	Rumusan Masalah	2
1.3	Batasan Masalah	3
1.4	Tujuan Penelitian	4
1.5	Manfaat Penelitian	4
BAB II	TINJAUAN PUSTAKA	6
2.1	Konsep Sistem Informasi	6
2.1.1	Definisi Sistem Informasi	6
2.1.2	Komponen Sistem Informasi	7
2.1.3	Fungsi Sistem Informasi	7
2.1.4	Sistem Informasi dalam Pendidikan	8
2.1.5	Keunggulan Sistem Informasi Terpusat	8
2.2	Manajemen Tugas dan Penilaian Praktikum	9
2.3	Rekayasa Perangkat Lunak	10
2.4	Metodologi Pengembangan (Waterfall)	13
2.5	UML	15
2.5.1	Use Case Diagram	15
2.5.2	Activity Diagram	16
2.5.3	Sequence Diagram	16
2.5.4	Class Diagram	16
2.5.5	Collaboration Diagram	17
2.5.6	State Diagram	17
2.6	Data Flow Diagram (DFD)	18
2.6.1	Komponen Utama DFD	20
2.6.2	Tingkatan Hierarki DFD	20
2.7	Entity Relationship Diagram (ERD)	20
2.8	UI/UX Design Principles	21
BAB III	METODOLOGI PENELITIAN	23
3.1	Metode Pengembangan Sistem	23
3.2	Tahapan Pengembangan	24
3.2.1	Planning (Perencanaan)	24
3.2.2	Analysis (Analisis)	24
3.2.3	Design (Perancangan)	24
3.2.4	Implementation (Implementasi)	24

3.2.5	Testing (Pengujian)	24
BAB IV	ANALISIS DAN PERANCANGAN SISTEM	25
4.1	Analisis Sistem	25
4.1.1	Analisis Sistem Berjalan (Flowmap)	25
4.1.2	Identifikasi Masalah	27
4.1.3	Analisis Kebutuhan Sistem	27
4.2	Perancangan Sistem	28
4.2.1	Diagram Konteks (DFD Level 0)	28
4.2.2	DFD Level 1	28
4.2.3	Perancangan Basis Data (ERD + Relasi Tabel)	28
4.2.4	Perancangan UML	28
4.3	Perancangan Antarmuka (UI/UX)	28
4.3.1	Wireframe / Mockup	28
4.3.2	Desain Tampilan	28
4.3.3	User Flow	28
BAB V	IMPLEMENTASI DAN PENGUJIAN	29
5.1	Implementasi	29
5.1.1	Implementasi Sistem (Frontend & Backend)	29
5.1.2	Implementasi Database	29
5.1.3	Screenshot Hasil Sistem	29
5.2	Pengujian	29
5.2.1	Metode Pengujian (Black Box)	29
5.2.2	Hasil Pengujian	29
5.2.3	Evaluasi Sistem	29
BAB VI	PENUTUP	30
6.1	Kesimpulan	30
6.2	Saran	30
Daftar Pustaka		31

DAFTAR GAMBAR

2.1	Tahapan SDLC	11
2.2	Tahapan model Waterfall	13
2.3	Simbol-simbol pada DFD	19
4.1	Flowmap Sistem Usulan EduLab	26

DAFTAR TABEL

BAB I

PENDAHULUAN

1.1 Latar Belakang

Praktikum merupakan komponen penting dalam proses pembelajaran di bidang ilmu komputer dan rekayasa perangkat lunak yang bertujuan untuk mengembangkan keterampilan praktis mahasiswa di samping pemahaman teoritis. Dalam konteks pendidikan tinggi, praktikum laboratorium berfungsi sebagai wadah aplikasi konsep-konsep akademis ke dalam kasus nyata, sehingga mahasiswa dapat mengasah kemampuan analisis, pemecahan masalah, dan kolaborasi dalam lingkungan yang mendekati kondisi profesional. Efektivitas proses praktikum tidak hanya bergantung pada kualitas materi dan instruktur, tetapi juga pada sistem manajemen yang mengatur pengumpulan tugas, proses penilaian, dokumentasi, dan umpan balik kepada mahasiswa.

Saat ini, manajemen tugas dan penilaian praktikum di banyak perguruan tinggi di Indonesia masih dilakukan secara manual atau menggunakan kombinasi alat digital sederhana seperti Excel, Google Drive, dan Google Form. Mahasiswa mengumpulkan tugas melalui file di Google Drive atau email, sedangkan asisten praktikum dan dosen merekap nilai dalam file Excel terpisah tanpa sistem terpusat. Proses ini seringkali dilakukan secara terpecah antara berbagai platform, sehingga tidak ada satu sumber informasi yang dapat diakses secara konsisten oleh semua pihak terkait. Ketergantungan pada alat-alat yang tidak terintegrasi menyebabkan data tersebar di berbagai lokasi dan sulit untuk dikelola secara efisien.

Kondisi tersebut menyebabkan beberapa permasalahan yang signifikan. Pertama, tidak adanya tracking otomatis status pengumpulan tugas sehingga tugas dapat terlewat atau tidak tercatat secara akurat. Dosen dan asisten praktikum seringkali harus melakukan pengecekan manual terhadap setiap mahasiswa untuk memastikan apakah tugas telah dikumpulkan, yang memakan waktu dan tidak efisien. Kedua, penilaian yang tidak terdokumentasi secara terpusat dan sulit diaudit. Nilai yang direkap dalam file Excel terpisah tidak memiliki riwayat yang jelas mengenai perubahan nilai, sehingga jika terjadi kesalahan atau pertanyaan dari mahasiswa, proses verifikasi menjadi sangat rumit.

Ketiga, umpan balik ke mahasiswa yang tidak terstruktur dan tersebar di berbagai chat atau file. Mahasiswa seringkali menerima feedback dalam bentuk komentar di file, pesan di chat pribadi, atau email, yang tidak terorganisir dalam satu tempat. Hal ini menyebabkan mahasiswa sulit untuk melacak perkembangan pembelajaran mereka dan mereferensi feedback sebelumnya untuk perbaikan di tugas berikutnya. Keempat, rekap nilai dan sta-

tistik yang harus dilakukan manual sehingga memakan waktu. Asisten praktikum dan dosen harus melakukan perhitungan dan rekap secara manual setiap kali diperlukan data statistik seperti rata-rata nilai, distribusi nilai, atau analisis performa per kelompok.

Selain itu, mahasiswa baru sering bingung memilih kelas atau kelompok karena menu yang tidak jelas. Sistem yang tidak terstruktur menyebabkan kesulitan navigasi bagi mahasiswa yang baru pertama kali menggunakan platform praktikum, sehingga mereka memerlukan bantuan tambahan dari asisten atau teman untuk memahami alur kerja yang benar. Hal ini menambah beban kerja asisten dan mengurangi waktu yang seharusnya dapat digunakan untuk akademik.

Kekurangan sistem yang ada menyebabkan beban kerja dosen dan asisten meningkat secara signifikan, karena mereka harus melakukan tugas administrasi yang berulang dan memakan waktu. Potensi ketidakadilan dalam penilaian juga meningkat karena tidak ada dokumentasi yang terpusat dan auditabel yang dapat memastikan konsistensi penilaian antar mahasiswa dan antar kelompok. Penurunan efisiensi pembelajaran juga terjadi akibat feedback yang tidak terstruktur, sehingga mahasiswa tidak mendapatkan umpan balik yang jelas dan dapat ditindaklanjuti untuk perbaikan.

Oleh karena itu, diperlukan sistem informasi yang terpusat untuk mengelola tugas dan penilaian praktikum secara lebih efektif. Sistem yang dirancang harus dapat mengatasi permasalahan tracking tugas, dokumentasi penilaian, umpan balik terstruktur, dan rekap otomatis. Sistem tersebut juga harus menyediakan antarmuka yang mudah digunakan untuk semua pihak, termasuk mahasiswa baru, sehingga dapat mengurangi kebingungan dan kebutuhan bantuan tambahan. EduLab dirancang dengan memperhitungkan kebutuhan-kebutuhan tersebut, dengan fokus pada kemudahan penggunaan, efektivitas manajemen data, dan kemampuan untuk meningkatkan kualitas pembelajaran praktikum.

Berdasarkan hal tersebut, penulis merancang EduLab: Sistem Manajemen Tugas dan Penilaian Praktikum yang dapat membantu dosen, asisten praktikum, dan mahasiswa dalam mengelola proses praktikum secara lebih terstruktur dan efisien. EduLab diharapkan dapat menjadi solusi untuk permasalahan yang ada saat ini, serta meningkatkan kualitas manajemen praktikum di laboratorium sehingga proses pembelajaran dapat berjalan lebih efektif dan efisien. Dengan adanya sistem terpusat, semua pihak terkait dapat mengakses informasi yang konsisten, melakukan tugas administrasi dengan lebih cepat, dan memberikan umpan balik yang lebih bermanfaat bagi perkembangan mahasiswa.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, rumusan masalah dalam penelitian ini adalah:

1. Bagaimana merancang sistem EduLab yang mampu memfasilitasi pengumpulan dan pelacakan tugas praktikum secara terpusat dan efisien?
2. Bagaimana mendokumentasikan proses penilaian dalam EduLab agar bersifat auditabel, transparan, dan konsisten?
3. Bagaimana merancang antarmuka umpan balik terstruktur dalam EduLab guna mendukung perkembangan belajar mahasiswa?
4. Bagaimana menerapkan sistem manajemen peran meliputi dosen, asisten praktikum, dan mahasiswa dalam platform EduLab?

1.3 Batasan Masalah

Agar penelitian ini lebih terarah dan terfokus, maka ditetapkan batasan-batasan masalah sebagai berikut:

1. Sistem yang dikembangkan hanya mencakup tiga peran pengguna, yaitu *mahasiswa*, *asisten praktikum*, dan *dosen*, dengan hak akses yang berbeda pada setiap peran.
2. Proses autentikasi dilakukan berdasarkan NIM untuk mahasiswa dan NIP untuk dosen dan asisten praktikum yang telah terdaftar dalam basis data pengguna (DB_Pengguna).
3. Fitur pengumpulan tugas hanya mendukung unggah berkas laporan dalam batas waktu (*deadline*) yang telah ditentukan; berkas yang diunggah melewati *deadline* akan ditolak secara otomatis oleh sistem.
4. Penilaian dilakukan oleh asisten praktikum melalui verifikasi dan evaluasi tugas, kemudian nilai disimpan ke basis data nilai (DB_Nilai) dan nilai akhir dihitung secara otomatis oleh sistem.
5. Mekanisme sanggah hanya dapat diajukan oleh mahasiswa dan ditinjau oleh asisten praktikum, dengan dua kemungkinan keputusan: *setuju* (nilai diperbarui) atau *tolak* (tanggapan penolakan disimpan).
6. Status kelulusan ditentukan berdasarkan perbandingan nilai akhir terhadap nilai minimal yang ditetapkan, dengan indikator visual berupa warna hijau untuk *lulus* dan warna merah untuk *mengulang*.
7. Pengelolaan kelas dan kelompok praktikum, termasuk unggah modul, sepenuhnya menjadi wewenang dosen.

8. Sistem menyediakan fitur rekap nilai per kelas yang dapat diekspor dalam bentuk berkas oleh dosen.
9. Sistem tidak mencakup fitur komunikasi langsung antara pengguna seperti *chat* atau forum diskusi.
10. Sistem dikembangkan sebagai aplikasi berbasis web dan tidak mencakup pengembangan aplikasi *mobile*.

1.4 Tujuan Penelitian

Berdasarkan rumusan masalah yang telah diuraikan, tujuan dari penelitian ini adalah:

1. Merancang sistem EduLab yang mampu memfasilitasi pengumpulan dan pelacakan tugas praktikum secara terpusat dan efisien.
2. Mendokumentasikan proses penilaian dalam EduLab agar bersifat auditabel, transparan, dan konsisten.
3. Merancang antarmuka umpan balik terstruktur dalam EduLab guna mendukung perkembangan belajar mahasiswa.
4. Menerapkan sistem manajemen peran meliputi dosen, asisten praktikum, dan mahasiswa dalam platform EduLab.

1.5 Manfaat Penelitian

Penelitian ini diharapkan memberikan manfaat bagi berbagai pihak, antara lain:

1. Bagi Mahasiswa
 - a. Memudahkan proses pengumpulan laporan tugas praktikum secara daring tanpa batasan tempat dan waktu.
 - b. Memberikan transparansi nilai melalui tampilan hasil penilaian dan status kelulusan secara langsung.
 - c. Menyediakan mekanisme sanggah yang terstruktur sehingga mahasiswa dapat mengajukan keberatan penilaian secara resmi dan terdokumentasi.

2. Bagi Asisten Praktikum

- a. Mempermudah proses verifikasi, evaluasi tugas, dan pemberian nilai melalui antarmuka yang terstruktur.
- b. Membantu pengelolaan presensi mahasiswa secara digital dan tersimpan dalam basis data secara otomatis.
- c. Mengurangi beban administratif melalui perhitungan nilai akhir yang dilakukan secara otomatis oleh sistem.

3. Bagi Dosen

- a. Memberikan kemudahan dalam pembuatan dan pengelolaan kelas serta kelompok praktikum.
- b. Menyediakan fitur unggah modul yang terpusat sehingga materi praktikum dapat diakses oleh seluruh peserta.
- c. Memfasilitasi monitoring perkembangan kelas dan ekspor rekapitulasi nilai per kelas sebagai bahan evaluasi.

4. Bagi Institusi

- a. Meningkatkan kualitas pengelolaan praktikum melalui sistem yang terdigitalisasi dan terintegrasi.
- b. Menghasilkan dokumentasi penilaian yang auditabel sebagai bentuk akuntabilitas akademik.
- c. Memberikan referensi pengembangan sistem informasi akademik berbasis manajemen peran di lingkungan perguruan tinggi.

BAB II

TINJAUAN PUSTAKA

2.1 Konsep Sistem Informasi

Sistem informasi merupakan kombinasi dari komponen manusia, perangkat keras, perangkat lunak, data, dan prosedur yang bekerja bersama untuk mengumpulkan, memproses, menyimpan, dan menyebarluaskan informasi guna mendukung pengambilan keputusan, koordinasi, kontrol, analisis, dan visualisasi dalam suatu organisasi [1]. Dalam konteks pendidikan, sistem informasi berperan penting untuk mengelola proses pembelajaran, administrasi, dan interaksi antara dosen, asisten praktikum, serta mahasiswa.

2.1.1 Definisi Sistem Informasi

Menurut Laudon dan Laudon, sistem informasi adalah kumpulan komponen yang saling berinteraksi yang bekerja untuk mengumpulkan, memproses, menyimpan, dan menyebarluaskan informasi untuk mendukung pengambilan keputusan dan kontrol dalam organisasi [1]. Sistem informasi tidak hanya berupa teknologi komputer, tetapi juga mencakup aspek manusia, prosedur, dan kebijakan yang mengatur penggunaan teknologi tersebut.

Sistem informasi dapat dikategorikan menjadi beberapa jenis berdasarkan fungsinya:

1. Transaction Processing Systems (TPS): Sistem yang mencatat dan memproses transaksi harian dalam organisasi, seperti sistem registrasi mahasiswa atau sistem pengumpulan tugas [2].
2. Management Information Systems (MIS): Sistem yang menyediakan laporan dan informasi untuk mendukung pengambilan keputusan tingkat menengah, seperti rekap nilai praktikum [2].
3. Decision Support Systems (DSS): Sistem yang membantu pengambilan keputusan melalui analisis data dan model, seperti analisis performa mahasiswa per kelompok [2].
4. Executive Support Systems (ESS): Sistem yang menyediakan informasi strategis untuk keputusan tingkat atas, seperti evaluasi kualitas praktikum secara keseluruhan [2].

2.1.2 Komponen Sistem Informasi

Sistem informasi terdiri dari lima komponen utama yang saling terkait:

1. Manusia (People): Pengguna sistem yang mencakup pemasok data, pemroses informasi, dan pengambil keputusan. Dalam EduLab, manusia terdiri dari dosen, asisten praktikum, dan mahasiswa [1].
2. Perangkat Keras (Hardware): Komponen fisik seperti komputer, server, smartphone, dan jaringan yang digunakan untuk mengakses sistem. EduLab berbasis web dapat diakses melalui berbagai perangkat keras [2].
3. Perangkat Lunak (Software): Aplikasi dan program yang menjalankan proses informasi, termasuk sistem operasi, database, dan aplikasi web. EduLab menggunakan PHP, framework modern, dan MySQL sebagai database [2].
4. Data: Informasi yang diolah dan disimpan dalam sistem. Dalam EduLab, data mencakup tugas mahasiswa, nilai, rubrik penilaian, dan umpan balik [1].
5. Prosedur (Procedures): Aturan, kebijakan, dan langkah-langkah yang mengatur penggunaan sistem. Dalam EduLab, prosedur mencakup alur pengumpulan tugas, proses penilaian, dan mekanisme umpan balik [2].

2.1.3 Fungsi Sistem Informasi

Sistem informasi memiliki beberapa fungsi utama dalam organisasi:

1. Pengumpulan Data: Mengambil data dari sumber internal atau eksternal, seperti pengumpulan tugas dari mahasiswa [1].
2. Pemrosesan Informasi: Mengubah data menjadi informasi yang lebih berguna melalui komputasi, analisis, atau klasifikasi, seperti perhitungan nilai berdasarkan rubrik [2].
3. Penyimpanan Data: Menyimpan informasi dalam database untuk akses selanjutnya, seperti penyimpanan nilai dan riwayat tugas [1].
4. Penyebaran Informasi: Distribusi informasi kepada pengguna yang membutuhkan, seperti menampilkan nilai dan umpan balik kepada mahasiswa [2].
5. Pengendalian dan Kontrol: Meng-monitor dan mengatur proses untuk memastikan sistem berjalan sesuai tujuan, seperti tracking status pengumpulan tugas [1].

2.1.4 Sistem Informasi dalam Pendidikan

Dalam konteks pendidikan, sistem informasi digunakan untuk mendukung berbagai aktivitas akademik, termasuk:

1. Manajemen registrasi dan administrasi mahasiswa [3]
2. Sistem pembelajaran elektronik (e-learning) [4]
3. Manajemen tugas dan penilaian [3]
4. Rekap dan analisis performa akademik [3]
5. Komunikasi antara dosen, asisten, dan mahasiswa [5]

Sistem informasi untuk praktikum laboratorium memiliki karakteristik khusus yang berbeda dari sistem akademik umum, seperti kebutuhan untuk tracking detail status tugas, dokumentasi penilaian yang auditabel, dan umpan balik terstruktur per tugas. EduLab dirancang untuk memenuhi kebutuhan khusus tersebut dengan fokus pada manajemen tugas dan penilaian praktikum secara terpusat.

2.1.5 Keunggulan Sistem Informasi Terpusat

Sistem informasi terpusat seperti EduLab memiliki beberapa keunggulan dibandingkan sistem yang tersebar:

1. Konsistensi Data: Semua pihak mengakses informasi yang sama dari satu sumber, mengurangi perbedaan data [2].
2. Efisiensi Akses: Tidak perlu berpindah antar platform untuk mengakses informasi yang berbeda [2].
3. Auditabilitas: Riwayat perubahan data dapat dilacak untuk verifikasi dan evaluasi [1].
4. Kolaborasi: Dosen, asisten, dan mahasiswa dapat bekerja dalam satu platform yang terintegrasi [2].
5. Skalabilitas: Sistem dapat dikembangkan untuk menambah fitur atau pengguna tanpa mengubah arsitektur dasar [1].

Dengan pemahaman tentang konsep sistem informasi di atas, EduLab dirancang sebagai sistem informasi terpusat yang mendukung manajemen tugas dan penilaian praktikum secara lebih efektif, efisien, dan terstruktur untuk semua pihak terkait.

2.2 Manajemen Tugas dan Penilaian Praktikum

Manajemen tugas dan penilaian praktikum merupakan proses pengelolaan seluruh aktivitas yang berkaitan dengan pemberian tugas, pengumpulan tugas, pemeriksaan hasil pekerjaan mahasiswa, pemberian nilai, serta penyampaian umpan balik dalam kegiatan praktikum. Proses ini memiliki peran penting dalam memastikan bahwa kegiatan praktikum berjalan secara terstruktur, transparan, dan sesuai dengan capaian pembelajaran yang telah ditetapkan. Dalam lingkungan pendidikan tinggi, khususnya pada mata kuliah yang bersifat aplikatif seperti pemrograman, basis data, rekayasa perangkat lunak, dan sistem informasi, manajemen tugas dan penilaian praktikum membantu dosen serta asisten praktikum dalam memantau perkembangan kemampuan mahasiswa secara lebih objektif dan berkelanjutan [6].

Secara umum, manajemen tugas praktikum mencakup beberapa tahapan utama, yaitu perencanaan tugas, pendistribusian tugas kepada mahasiswa, pengumpulan hasil pengerjaan, verifikasi kelengkapan, serta pencatatan status pengumpulan. Pada tahap perencanaan, dosen atau asisten praktikum menentukan jenis tugas yang diberikan, tujuan pembelajaran yang ingin dicapai, batas waktu pengumpulan, serta format output yang diharapkan. Setelah itu, tugas didistribusikan kepada mahasiswa melalui media tertentu, baik secara langsung maupun melalui sistem informasi. Mahasiswa kemudian mengerjakan tugas sesuai dengan instruksi yang diberikan dan mengunggah hasilnya sebelum tenggat waktu yang telah ditentukan.

Tahap berikutnya adalah penilaian praktikum, yaitu proses evaluasi terhadap hasil tugas mahasiswa berdasarkan kriteria tertentu. Penilaian ini dapat dilakukan menggunakan rubrik yang mencakup aspek ketepatan jawaban, kelengkapan isi, kualitas implementasi, kerapian laporan, dan ketepatan waktu pengumpulan. Dengan adanya rubrik penilaian, proses evaluasi menjadi lebih konsisten karena setiap mahasiswa dinilai berdasarkan indikator yang sama [7]. Selain itu, rubrik juga membantu dosen dan asisten praktikum dalam memberikan penilaian yang lebih objektif serta meminimalkan unsur subjektivitas dalam proses evaluasi.

Dalam praktiknya, penilaian praktikum tidak hanya menghasilkan angka atau nilai akhir, tetapi juga melibatkan pemberian umpan balik kepada mahasiswa. Umpan balik ini sangat penting agar mahasiswa mengetahui kekurangan dari hasil pekerjaannya dan dapat melakukan perbaikan pada tugas berikutnya [8]. Umpan balik yang baik seharusnya diberikan secara jelas, spesifik, dan mudah dipahami, sehingga dapat mendukung proses pembelajaran yang berkelanjutan. Pada sistem manual, umpan balik sering kali diberikan melalui pesan pribadi, komentar terpisah, atau catatan di file yang berbeda, sehingga sulit untuk dilacak kembali. Oleh karena itu, sistem yang terpusat diperlukan agar proses pemberian umpan balik menjadi lebih efektif dan terdokumentasi dengan baik.

Manajemen tugas dan penilaian praktikum juga berkaitan erat dengan pencatatan data akademik. Setiap tugas yang dikumpulkan, setiap nilai yang diberikan, dan setiap komentar yang disampaikan perlu disimpan secara sistematis agar dapat digunakan sebagai bahan rekapitulasi maupun evaluasi pembelajaran. Pencatatan yang baik akan memudahkan dosen dan asisten praktikum dalam melihat perkembangan mahasiswa, membandingkan hasil antarpertemuan, serta menyusun laporan akhir praktikum. Selain itu, data yang tersimpan dengan baik juga bermanfaat apabila terjadi perbedaan persepsi antara mahasiswa dan pengajar terkait hasil penilaian [9].

Pada sistem yang belum terintegrasi, manajemen tugas dan penilaian praktikum sering menghadapi berbagai kendala, seperti keterlambatan pencatatan nilai, hilangnya data tugas, duplikasi file, serta kesulitan dalam melacak status pengumpulan. Kondisi ini dapat menghambat efektivitas kerja dosen dan asisten praktikum, serta menurunkan pengalaman belajar mahasiswa. Oleh sebab itu, dibutuhkan sistem informasi yang mampu mengotomatisasi sebagian proses administrasi, menyediakan penyimpanan data yang terpusat, dan mendukung proses evaluasi secara lebih cepat dan akurat [10].

Berdasarkan hal tersebut, manajemen tugas dan penilaian praktikum dalam penelitian ini dipahami sebagai proses pengelolaan tugas dan evaluasi hasil praktikum yang dilakukan secara terstruktur melalui sistem informasi. EduLab dirancang untuk mendukung proses tersebut dengan menyediakan fitur pengumpulan tugas, penilaian berbasis rubrik, pencatatan nilai, serta pemberian umpan balik dalam satu platform yang terintegrasi. Dengan adanya sistem ini, diharapkan proses praktikum dapat berjalan lebih efisien, transparan, dan mudah dipantau oleh seluruh pihak yang terlibat.

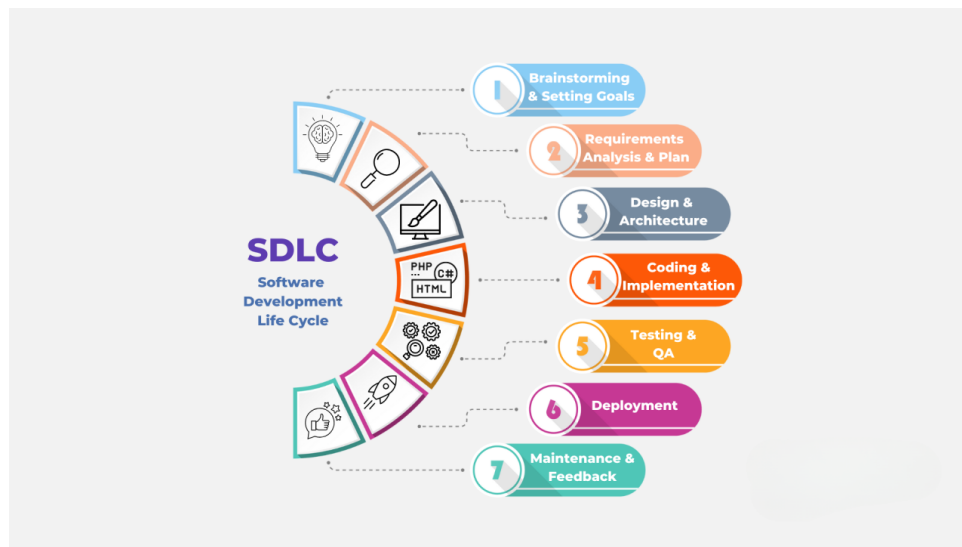
2.3 Rekayasa Perangkat Lunak

Rekayasa perangkat lunak (*software engineering*) merupakan disiplin ilmu yang berkaitan dengan penerapan pendekatan sistematis, terstruktur, dan terukur dalam pengembangan, pengoperasian, serta pemeliharaan perangkat lunak [11]. Disiplin ini tidak hanya berfokus pada aspek teknis pemrograman, tetapi juga mencakup berbagai aktivitas penting seperti manajemen proyek, analisis kebutuhan, perancangan sistem, pengujian, dokumentasi, implementasi, serta pemeliharaan. Seluruh proses tersebut bertujuan untuk menghasilkan perangkat lunak yang memiliki kualitas tinggi dan mampu memenuhi kebutuhan pengguna secara optimal.

Menurut Pressman, rekayasa perangkat lunak dapat dipandang sebagai teknologi berlapis yang terdiri atas proses (*process*), metode (*methods*), alat bantu (*tools*), dan fokus pada kualitas (*quality focus*) [12]. Keempat lapisan ini saling berhubungan dan bekerja secara terpadu dalam mendukung pengembangan perangkat lunak yang efektif. Proses menjadi kerangka kerja utama yang mengatur tahapan pengembangan, metode menyedi-

akan teknik-teknik yang digunakan dalam setiap tahap, alat bantu mendukung otomatisasi dan efisiensi kerja, sedangkan fokus kualitas memastikan bahwa hasil akhir sesuai dengan standar yang diharapkan. Pressman juga menekankan bahwa rekayasa perangkat lunak merupakan penerapan prinsip-prinsip rekayasa untuk menghasilkan perangkat lunak yang andal dan efisien secara ekonomis, sehingga pendekatan yang digunakan harus terstruktur dan terukur, bukan sekadar aktivitas pengkodean.

Tujuan utama dari rekayasa perangkat lunak adalah menghasilkan perangkat lunak yang memenuhi standar kualitas tertentu dalam batas waktu dan anggaran yang telah ditetapkan [11]. Untuk mencapai tujuan tersebut, proses pengembangan perangkat lunak umumnya dilakukan melalui beberapa tahapan yang saling berkaitan. Tahap pertama adalah *brainstorming* dan penetapan tujuan, yaitu proses awal untuk mengidentifikasi permasalahan, menentukan arah pengembangan, dan menetapkan sasaran sistem yang akan dibangun. Tahap kedua adalah analisis kebutuhan dan perencanaan, yang berfokus pada identifikasi, dokumentasi, serta validasi kebutuhan pengguna dan sistem. Dalam konteks pengembangan EduLab, tahap ini mencakup kebutuhan fungsional seperti fitur pengumpulan tugas dan penilaian, serta kebutuhan non-fungsional seperti keamanan dan performa sistem.



Gambar 2.1: Tahapan SDLC

Tahap berikutnya adalah perancangan sistem dan arsitektur yang bertujuan untuk mendefinisikan struktur sistem, komponen, antarmuka, dan karakteristik aplikasi secara menyeluruh [12]. Pada sistem EduLab, perancangan dilakukan melalui pembuatan berbagai model seperti DFD, ERD, diagram UML, serta desain antarmuka pengguna. Setelah perancangan selesai, proses dilanjutkan dengan coding dan implementasi, yaitu tahap pengkodean berdasarkan desain yang telah dibuat [11]. Implementasi EduLab dilakuk-

an menggunakan bahasa pemrograman PHP dengan dukungan framework modern serta MySQL sebagai basis data.

Selanjutnya dilakukan tahap *testing* dan *quality assurance* untuk memastikan bahwa perangkat lunak yang dikembangkan telah sesuai dengan kebutuhan yang ditentukan [12]. Pada pengembangan EduLab, pengujian dilakukan menggunakan metode *blackbox testing* untuk memverifikasi bahwa setiap fungsi berjalan dengan benar. Setelah sistem dinyatakan layak, tahap berikutnya adalah *deployment*, yaitu proses penerapan sistem ke lingkungan pengguna agar dapat digunakan secara nyata. Tahap terakhir adalah pemeliharaan dan umpan balik, yang mencakup perbaikan kesalahan, peningkatan kinerja, serta penyesuaian sistem berdasarkan masukan pengguna [11].

Secara umum, tahapan-tahapan tersebut menunjukkan bahwa rekayasa perangkat lunak bukan hanya kegiatan menulis kode, tetapi merupakan proses yang mencakup seluruh siklus hidup perangkat lunak dari perencanaan hingga pemeliharaan. Dalam konteks aplikasi berbasis web, pendekatan ini menjadi sangat penting karena sistem harus mampu diakses dari berbagai perangkat, menangani banyak pengguna secara bersamaan, memiliki skalabilitas yang baik, serta menyediakan antarmuka yang responsif dan mudah digunakan [12]. Penerapan prinsip-prinsip ini pada EduLab bertujuan untuk memastikan bahwa sistem yang dihasilkan memiliki keandalan tinggi, mudah dipelihara, dan efektif dalam mendukung proses pembelajaran serta evaluasi akademik. Selain itu, penerapan rekayasa perangkat lunak berbasis web secara terstruktur juga terbukti mampu meningkatkan kepuasan pengguna, kualitas penggunaan, dan efektivitas evaluasi pendidikan [13].

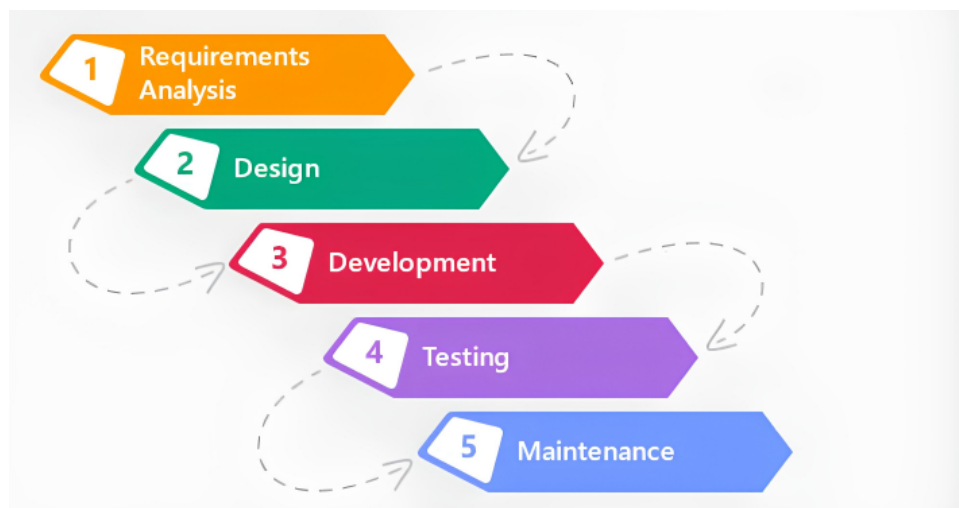
Kualitas perangkat lunak merupakan aspek penting yang mencerminkan tingkat kesesuaian antara perangkat lunak dengan kebutuhan yang telah ditetapkan [12]. Salah satu model awal dalam pengukuran kualitas adalah model McCall yang mengelompokkan kualitas ke dalam tiga perspektif, yaitu operasi produk, revisi produk, dan transisi produk. Perspektif operasi berkaitan dengan kemampuan sistem dalam menjalankan fungsinya secara benar dan efisien, revisi berfokus pada kemudahan dalam pemeliharaan dan pengembangan lebih lanjut, sedangkan transisi berkaitan dengan kemampuan perangkat lunak untuk beradaptasi dengan lingkungan yang berbeda.

Seiring perkembangan teknologi, evaluasi kualitas perangkat lunak mengacu pada standar yang lebih komprehensif seperti ISO/IEC 25010. Standar ini mencakup berbagai karakteristik kualitas yang lebih luas, antara lain *functional suitability*, *performance efficiency*, *compatibility*, *usability*, *reliability*, *security*, *maintainability*, dan *portability* [14]. Pendekatan ini memungkinkan pengukuran kualitas perangkat lunak secara lebih sistematis dan terukur sesuai dengan kebutuhan modern.

2.4 Metodologi Pengembangan (Waterfall)

Model Waterfall adalah salah satu metodologi pengembangan perangkat lunak yang paling klasik dan banyak digunakan dalam rekayasa perangkat lunak tradisional. Model ini pertama kali diperkenalkan oleh Winston W. Royce pada tahun 1970, meskipun istilah "Waterfall" sendiri kemudian dipopulerkan oleh Benjamin Weingrad pada tahun 1976. Model Waterfall menggunakan pendekatan sekuensial-linear di mana setiap fase pengembangan harus selesai sepenuhnya sebelum fase berikutnya dimulai, mirip dengan aliran air yang jatuh dari atas ke bawah[15].

Model Waterfall adalah pendekatan pengembangan perangkat lunak yang membagi proses pengembangan dalam serangkaian fase yang berurutan dan tidak tumpang tindih, di mana setiap fase menghasilkan output yang menjadi input untuk fase berikutnya[12]. Pendekatan ini bersifat disiplin dan terstruktur, sehingga memudahkan manajemen proyek dalam memantau progress dan mengontrol kualitas. Menurut Sommerville, model Waterfall membagi proses pengembangan perangkat lunak dalam fase-fase yang jelas dan terpisah, dengan setiap fase memiliki tujuan dan hasil yang spesifik. Keunggulan utama dari pendekatan ini adalah kemudahan dalam pengelolaan dan dokumentasi yang sistematis, namun kekurangannya adalah sulit untuk mengakomodasi perubahan kebutuhan di tengah proses pengembangan.



Gambar 2.2: Tahapan model Waterfall

Seperti ditunjukkan pada Gambar 2.2, model Waterfall terdiri dari enam fase utama yang dijalankan secara berurutan. Tahap pertama adalah Analisis Kebutuhan, yaitu fase awal dalam pengembangan perangkat lunak di mana kebutuhan pengguna dan sistem diidentifikasi, dianalisis, dan didokumentasikan secara lengkap. Dalam pengembangan EduLab, fase ini mencakup identifikasi kebutuhan fungsional seperti fitur pengumpulan

tugas, pelacakan status, penilaian berbasis rubrik, dan umpan balik terstruktur, serta kebutuhan non-fungsional seperti keamanan, performa, dan kemudahan penggunaan. Tahap kedua adalah Perancangan Sistem, yaitu setelah kebutuhan ditentukan, fase ini mendefinisikan arsitektur sistem, komponen, antarmuka, dan karakteristik lain yang diperlukan untuk memenuhi kebutuhan yang telah diidentifikasi. Dalam EduLab, perancangan sistem mencakup pembuatan DFD Level 0 dan Level 1, ERD untuk basis data, diagram UML (Use Case, Activity, Sequence, Class, Collaboration), serta desain antarmuka pengguna (UI/UX).

Tahap ketiga adalah Implementasi, yaitu proses pengkodean dan pengembangan sistem berdasarkan hasil perancangan yang telah dibuat. Pada EduLab, implementasi dilakukan menggunakan bahasa pemrograman PHP, framework modern untuk pengembangan web, dan MySQL sebagai basis data. Kode program ditulis sesuai dengan desain sistem yang telah dibuat sebelumnya. Tahap keempat adalah Pengujian, yaitu setelah implementasi selesai, sistem diuji untuk memastikan bahwa perangkat lunak yang dikembangkan berfungsi sesuai dengan kebutuhan yang telah ditentukan. Pengujian EduLab dilakukan menggunakan metode blackbox testing untuk memverifikasi bahwa setiap fitur seperti pengumpulan tugas, penilaian, dan umpan balik berjalan sesuai harapan tanpa error.

Tahap kelima adalah Penerapan, yaitu sistem yang telah diuji dan divalidasi kemudian diterapkan ke lingkungan produksi untuk digunakan oleh pengguna nyata. Dalam konteks EduLab, penerapan mencakup instalasi sistem di server kampus, konfigurasi database, dan pelatihan pengguna (dosen, asisten praktikum, dan mahasiswa) untuk menggunakan sistem. Tahap terakhir adalah Pemeliharaan, yaitu setelah sistem diterapkan, fase ini dilakukan untuk memperbaiki kesalahan yang terdeteksi, meningkatkan kinerja, atau menyesuaikan sistem dengan perubahan kebutuhan pengguna. Pemeliharaan EduLab mencakup perbaikan bug, penambahan fitur baru, dan optimasi performa sistem berdasarkan feedback pengguna.

Model Waterfall memiliki beberapa karakteristik utama yang membedakannya dari model pengembangan perangkat lunak lainnya. Pertama adalah pendekatan sekuensial di mana setiap fase harus selesai sepenuhnya sebelum fase berikutnya dimulai, tidak ada overlap atau iterasi. Kedua adalah dokumentasi lengkap di mana setiap fase menghasilkan dokumentasi yang lengkap dan formal, yang menjadi dasar untuk fase berikutnya. Ketiga adalah keteraturan dan disiplin di mana proses development terstruktur dan mudah dipantau oleh manajer proyek. Keempat adalah sulit mengakomodasi perubahan di mana perubahan kebutuhan di tengah proses pengembangan sulit dilaksanakan tanpa mengulang fase sebelumnya.

Keunggulan utama model Waterfall adalah kemudahan dalam pengelolaan proyek karena alur yang jelas dan dokumentasi yang lengkap. Namun, kekurangan utamanya adalah

sulit untuk mengakomodasi perubahan kebutuhan di tengah proses pengembangan, yang dapat menjadi masalah jika kebutuhan pengguna tidak sepenuhnya jelas di awal proyek. Model Waterfall juga tidak menghasilkan produk yang dapat digunakan sampai fase akhir, sehingga pengguna tidak dapat melihat hasil hingga deployment, dan risiko tinggi jika ada kesalahan di fase awal karena tidak ada mekanisme untuk koreksi mundur.

2.5 UML

Unified Modeling Language (UML) adalah bahasa standar visual untuk spesifikasi, konstruksi, dokumentasi, dan komunikasi elemen-elemen sistem berbasis objek [16]. UML diciptakan oleh Object Management Group (OMG) dengan versi awal 1.0 pada Januari 1997 dan sejak saat itu telah berevolusi menjadi standar internasional (ISO/IEC 19505) untuk pemodelan arsitektur perangkat lunak [17]. Dalam rekayasa perangkat lunak modern, penggunaan UML tidak hanya terbatas pada sistem berorientasi objek tradisional, tetapi juga diadaptasi secara luas untuk merancang arsitektur aplikasi berbasis web dan sistem manajemen basis data kompleks [18]. Pada pengembangan EduLab, UML digunakan sebagai instrumen utama untuk memodelkan perilaku dinamis sistem, menetapkan struktur data, mendefinisikan interaksi antar-objek, dan merancang alur aktivitas secara visual, sehingga menjembatani kesenjangan antara fase perancangan konseptual dan implementasi kode pemrograman.

2.5.1 Use Case Diagram

Use Case Diagram merupakan salah satu diagram perilaku dalam UML yang berfungsi untuk menggambarkan fungsionalitas sistem dari perspektif interaksi antara aktor (pengguna atau entitas eksternal) dengan sistem itu sendiri guna mencapai tujuan spesifik [19]. Diagram ini berperan sebagai fondasi untuk tahapan pengembangan selanjutnya karena memvisualisasikan apa yang seharusnya dilakukan oleh sistem tanpa menjelaskan detail teknis implementasinya [16]. Dalam konteks EduLab, diagram ini menetapkan aktor-aktor utama yang terdiri dari Dosen, Asisten Praktikum, dan Mahasiswa. Aktor-aktor tersebut dihubungkan dengan berbagai *use case* krusial seperti proses login, pengumpulan tugas, pelacakan status pengerjaan, pelaksanaan penilaian berbasis rubrik, hingga pemberian umpan balik terstruktur. Melalui identifikasi fungsionalitas ini, pengembang dapat memastikan bahwa seluruh kebutuhan operasional laboratorium akademik telah terakomodasi dalam skenario sistem [5].

2.5.2 Activity Diagram

Sebagai kelanjutan dari Use Case, Activity Diagram digunakan untuk menggambarkan aspek dinamis dari sistem dengan memperlihatkan aliran kontrol dari satu aktivitas ke aktivitas lainnya secara berurutan [19]. Secara visual, diagram ini menyerupai *flowchart* tingkat lanjut yang tidak hanya memetakan proses sekuensial, tetapi juga mampu merepresentasikan proses percabangan (*decision*), penggabungan (*merge*), serta eksekusi paralel yang berjalan secara bersamaan di dalam sistem [17]. Pada sistem EduLab, diagram aktivitas dirancang untuk mengurai logika operasional pada setiap *use case*. Misalnya, diagram ini memetakan langkah demi langkah alur pengumpulan tugas yang dimulai dari mahasiswa mengunggah berkas tugas, proses validasi oleh sistem, hingga proses pendistribusian status penyerahan. Demikian pula pada alur penilaian, diagram ini mendeskripsikan transisi dari saat asisten membuka dokumen tugas, memberikan skor berdasarkan kriteria rubrik, hingga sistem mengkalkulasi dan mempublikasikan umpan balik (*feedback*) kepada mahasiswa.

2.5.3 Sequence Diagram

Untuk memodelkan dinamika interaksi yang lebih spesifik, Sequence Diagram digunakan dengan fokus utama pada urutan waktu (*chronological order*) terjadinya pertukaran pesan antar-objek [17]. Diagram interaksi ini memetakan objek-objek yang berpartisipasi dalam sebuah *use case* dan secara detail menampilkan rentetan pesan (*messages*) yang dikirim dan diterima seiring berjalannya waktu eksekusi [19]. Idealnya, setiap skenario utama dalam sistem memiliki diagram sekuensnya masing-masing. Dalam perancangan EduLab, Sequence Diagram mengilustrasikan bagaimana aktor seperti Mahasiswa, Dosen, atau Asisten berinteraksi dengan komponen antarmuka, *controller*, dan entitas basis data. Contohnya, pada proses pengumpulan tugas, diagram ini memperlihatkan urutan komunikasi mulai dari antarmuka login, proses *Upload Tugas*, manipulasi data pada *Sistem Penilaian*, hingga instruksi penyimpanan ke dalam *Database Dashboard*, sehingga memberikan panduan teknis yang presisi bagi pemrogram saat fase implementasi [20].

2.5.4 Class Diagram

Class Diagram adalah diagram struktural paling fundamental dalam UML yang berfungsi untuk memodelkan arsitektur statis dari sebuah sistem dengan mendeskripsikan kelas, atribut, operasi (metode), serta berbagai jenis relasi antar-kelas tersebut [16]. Diagram ini memberikan cetak biru dari skema basis data dan struktur pemrograman berorientasi objek yang akan dibangun [19]. Dalam arsitektur EduLab, Class Diagram merangkum entitas-entitas utama yang meliputi kelas *User* sebagai induk (dengan atribut ID,

nama, email, *password*), yang kemudian berelasi dengan kelas spesifik seperti *Mahasiswa* (atribut ID, nama, kelas, kelompok), *Dosen* (atribut ID, nama, mata kuliah), dan *Asisten* (atribut ID, nama, mata kuliah yang diampu). Selain itu, terdapat kelas *Tugas* (atribut ID, judul, deskripsi, *deadline*), *Penilaian* (atribut ID, tugas ID, nilai, *feedback*), dan *Rubrik* (atribut ID, penilaian ID, kriteria, skor). Diagram ini juga memvisualisasikan kardinalitas relasi, seperti relasi *one-to-many* antara Dosen dan Tugas, *many-to-one* antara Mahasiswa dan Tugas, serta asosiasi *one-to-many* antara Tugas dan Penilaian, yang secara langsung merepresentasikan arsitektur basis data relasional MySQL yang digunakan.

2.5.5 Collaboration Diagram

Collaboration Diagram, yang pada spesifikasi UML versi 2.x ke atas lebih dikenal sebagai Communication Diagram, merupakan bentuk diagram interaksi lain yang menekankan pada organisasi struktural objek-objek yang berpartisipasi dalam suatu pertukaran pesan [17]. Berbeda dengan Sequence Diagram yang berfokus pada dimensi waktu, Communication Diagram merupakan perluasan dari Object Diagram yang menonjolkan bagaimana topologi dan asosiasi antar-objek terbentuk saat sistem merespons sebuah perintah [16]. Pada pemodelan EduLab, diagram ini diaplikasikan untuk memvisualisasikan jalur komunikasi antar-komponen, seperti interaksi dan asosiasi antara objek *Student*, *RegistrationController*, *CourseCatalogSystem*, dan *Course* dalam skenario registrasi kelas praktikum atau alur pengumpulan tugas. Setiap pesan yang melintasi relasi antar-objek ditandai dengan nomor sekuens hierarkis (misalnya nomor 1 untuk pesan tertinggi, lalu 1.1, 1.2 untuk *prefiks* di level yang sama) untuk mengindikasikan urutan eksekusinya, sehingga proses aliran data dapat dianalisis secara menyeluruh.

2.5.6 State Diagram

State Machine Diagram, atau yang sering disebut Statechart Diagram, adalah diagram yang mengilustrasikan keadaan (*state*) yang dapat dicapai oleh sebuah objek dinamis, serta transisi antar-keadaan tersebut yang dipicu oleh suatu peristiwa (*event*) [17]. Pemodelan ini sangat penting untuk objek-objek yang perilakunya berubah secara signifikan tergantung pada status spesifiknya pada waktu tertentu [19]. Di dalam ekosistem EduLab, State Diagram digunakan secara krusial untuk melacak siklus hidup (*lifecycle*) dari objek *Tugas* dan objek *Penilaian*. Untuk objek *Tugas*, diagram memetakan transisi mulai dari keadaan *Belum Pengumpulan*, berubah menjadi *Sudah Pengumpulan* saat mahasiswa mengunggah hasil kerjanya, beralih ke *Dalam Penilaian* saat asisten mengakses berkas tersebut, dan berakhir pada keadaan *Sudah Dinilai* atau *Revisi*. Demikian pula pada objek *Penilaian*, transisinya dilacak dari *Belum*, *Dalam Proses*, hingga status *Final*. Melalui pendefinisian *state* ini, sistem dapat merespons sebab dan akibat (*causes and effects*) dari

aktivitas pengguna dengan logika operasional yang ketat [20].

2.6 Data Flow Diagram (DFD)

Data Flow Diagram (DFD) adalah instrumen pemodelan visual yang digunakan dalam analisis dan perancangan sistem untuk merepresentasikan aliran data yang melewati sebuah sistem informasi [21]. Berbeda dengan *flowchart* yang menitikberatkan pada urutan kendali atau sekuens eksekusi program, DFD berfokus secara eksklusif pada pergerakan data dari dan ke dalam sistem beserta transformasinya. Menurut Kendall dan Kendall, DFD menyediakan model konseptual yang kuat dengan memvisualisasikan bagaimana sebuah sistem memproses data dari sudut pandang masukan (*input*) dan keluaran (*output*) [21]. Dengan menggunakan notasi grafis yang terstandarisasi dan minim jargon teknis, diagram aliran data ini menjadi alat komunikasi yang efektif untuk menjembatani pemahaman antara analis sistem, tim pengembang perangkat lunak, dan pengguna akhir mengenai bagaimana arsitektur informasi seharusnya beroperasi.

Secara konseptual, pembangunan sebuah DFD selalu bertumpu pada empat komponen dasar. Komponen pertama adalah entitas eksternal (*external entity*), yang merepresentasikan sumber asal atau tujuan akhir dari sebuah data yang berada di luar batasan sistem perangkat lunak yang sedang dibangun. Komponen kedua adalah proses (*process*), yang melambangkan suatu aktivitas operasional atau transformasi logika yang mengubah aliran data masukan menjadi aliran data keluaran. Komponen ketiga adalah penyimpanan data (*data store*), yang merujuk pada data yang sedang tidak bergerak atau repositori penyimpanan persisten, seperti tabel dalam basis data relasional. Komponen terakhir adalah aliran data (*data flow*), yang divisualisasikan dengan simbol anak panah terarah untuk menunjukkan jalur pergerakan informasi dari satu komponen ke komponen lainnya [21].

Seperti ditunjukkan pada Gambar 2.3, simbol DFD terdiri atas entitas eksternal, proses, aliran data, dan penyimpanan data untuk mengelola kompleksitas perancangan arsitektur perangkat lunak, DFD diorganisasikan secara hierarkis dan dipecah melalui metode dekomposisi *top-down*. Tingkatan tertinggi dalam hierarki ini dikenal sebagai Diagram Konteks (*Context Diagram*) atau DFD Level 0, yang menggambarkan keseluruhan sistem sebagai satu proses tunggal yang berinteraksi langsung dengan berbagai entitas eksternal di sekitarnya. Diagram Konteks berfungsi secara krusial untuk menetapkan batasan sistem (*system boundary*) dan mengidentifikasi antarmuka pertukaran data dengan dunia luar.

Tingkatan selanjutnya adalah DFD Level 1, yang memecah proses tunggal dari Diagram Konteks menjadi beberapa subproses fungsional utama, serta mulai memperlihatkan entitas penyimpanan data inti dan aliran data internal sistem. Jika spesifikasi teknis membutuhkan rincian yang lebih mendalam, dekomposisi tersebut dapat diteruskan ke DFD

Keterangan	DeMarco and Yourdan Simbol	Gane and Sarson Simbol
Entitas Luar		
Proses		
Aliran data (data flow)		
Simpan data		

Gambar 2.3: Simbol-simbol pada DFD

Level 2 atau Level 3, yang secara spesifik memecah proses-proses di Level 1 menjadi transformasi logika dasar hingga setiap fungsionalitas siap didokumentasikan ke dalam bentuk kode [21]. Penerapan pemetaan aliran data yang terstruktur menggunakan DFD ini tidak hanya menjamin seluruh prasyarat fungsional sistem terpenuhi, tetapi juga membantu menjaga konsistensi integritas data di seluruh siklus hidup transaksi. Sebagai sebuah pendekatan *engineering*, penggunaan DFD terbukti mampu meminimalkan redundansi arsitektur dan meningkatkan efisiensi pelacakan aliran data, khususnya pada pengembangan sistem informasi berbasis web yang kompleks [5].

Data Flow Diagram (DFD) adalah instrumen pemodelan visual yang digunakan dalam analisis dan perancangan sistem untuk merepresentasikan aliran data yang melewati sebuah sistem informasi [21]. Berbeda dengan *flowchart* yang menitikberatkan pada urutan kendali atau sekuens eksekusi program, DFD berfokus secara eksklusif pada pergerakan data dari dan ke dalam sistem beserta transformasinya. Menurut Kendall dan Kendall, DFD menyediakan model konseptual yang tangguh dengan memvisualisasikan bagaimana sebuah sistem memproses data dari sudut pandang masukan (*input*) dan keluaran (*output*) [21]. Dengan menggunakan notasi grafis yang terstandarisasi dan minim jargon teknis, diagram aliran data ini menjadi alat komunikasi yang sangat efektif untuk menjembatani pemahaman antara analis sistem, tim pengembang perangkat lunak, dan pengguna akhir mengenai bagaimana arsitektur informasi seharusnya beroperasi.

2.6.1 Komponen Utama DFD

Secara konseptual, pembangunan sebuah DFD—baik menggunakan pendekatan notasi Gane-Sarson maupun Yourdon-DeMarco selalu bertumpu pada empat komponen dasar. Komponen pertama adalah Entitas Eksternal (*External Entity*), yang merepresentasikan sumber asal atau tujuan akhir dari sebuah data yang berada di luar batasan sistem perangkat lunak yang sedang dibangun. Komponen kedua adalah Proses (*Process*), yang melambangkan suatu aktivitas operasional, manipulasi, atau transformasi logika yang mengubah aliran data masukan menjadi aliran data keluaran. Komponen ketiga adalah Penyimpanan Data (*Data Store*), yang merujuk pada data yang sedang tidak bergerak (*data at rest*) atau repositori penyimpanan persisten, seperti tabel dalam basis data relasional. Komponen terakhir adalah Aliran Data (*Data Flow*), yang divisualisasikan dengan simbol anak panah terarah untuk menunjukkan jalur pergerakan atau transmisi informasi dari satu entitas, proses, atau penyimpanan data ke komponen lainnya [21].

2.6.2 Tingkatan Hierarki DFD

Untuk mengelola kompleksitas perancangan arsitektur perangkat lunak yang masif, DFD diorganisasikan secara hierarkis dan dipecah melalui metode dekomposisi *top-down*. Tingkatan tertinggi dalam hierarki ini dikenal sebagai Diagram Konteks (*Context Diagram*) atau DFD Level 0, yang menggambarkan keseluruhan sistem sebagai satu proses tunggal yang berinteraksi langsung dengan berbagai entitas eksternal di sekitarnya. Diagram Konteks berfungsi secara krusial untuk menetapkan batasan sistem (*system boundary*) dan mengidentifikasi antarmuka pertukaran data dengan dunia luar. Tingkatan selanjutnya adalah DFD Level 1, yang memecah proses tunggal dari Diagram Konteks menjadi beberapa sub-proses fungsional utama, serta mulai memperlihatkan entitas penyimpanan data inti dan aliran data internal sistem. Jika spesifikasi teknis membutuhkan rincian yang lebih mendalam, dekomposisi tersebut dapat diteruskan ke DFD Level 2 atau Level 3, yang secara spesifik memecah proses-proses di Level 1 menjadi transformasi logika dasar (*primitive process*) hingga setiap fungsionalitas siap didokumentasikan ke dalam bentuk kode [21].

2.7 Entity Relationship Diagram (ERD)

Entity Relationship Diagram (ERD) merupakan sebuah instrumen pemodelan data konseptual yang digunakan untuk merancang, menganalisis, dan merepresentasikan struktur logis dari sebuah pangkalan data [22]. Pemodelan ini pertama kali diperkenalkan oleh Peter Chen pada tahun 1976 sebagai pendekatan terpadu untuk mendeskripsikan data di dunia nyata dalam bentuk entitas dan hubungan matematisnya [23]. Dalam rekayasa per-

angkat lunak, ERD berfungsi sebagai cetak biru visual yang memfasilitasi komunikasi antara perancang basis data, pengembang sistem, dan pemangku kepentingan mengenai aturan bisnis serta batasan data sebelum sistem tersebut diimplementasikan secara fisik ke dalam *Database Management System* (DBMS). Melalui pendekatan grafis yang representatif, ERD membantu memetakan kompleksitas informasi menjadi struktur relasional yang lebih terorganisir, konsisten, dan bebas dari anomali struktural.

Secara fundamental, arsitektur logis dari sebuah ERD dibangun di atas tiga komponen pembentuk utama, yaitu entitas, atribut, dan relasi. Entitas merepresentasikan sekumpulan objek, konsep, atau peristiwa dunia nyata yang keberadaannya bersifat unik dan dapat diidentifikasi secara independen di dalam lingkup sistem [24]. Setiap entitas memiliki sekumpulan karakteristik spesifik yang disebut sebagai atribut, yang berfungsi untuk mendeskripsikan sifat, properti, atau detail informasi dari entitas tersebut. Di antara sekumpulan atribut tersebut, terdapat atribut kunci (*primary key*) yang bertugas secara spesifik untuk membedakan setiap baris data secara unik di dalam sebuah entitas. Sementara itu, relasi menggambarkan ikatan asosiatif tentang bagaimana dua atau lebih entitas saling berinteraksi satu sama lain. Relasi ini selalu diiringi dengan batasan kardinalitas yang mendefinisikan rasio pemetaan kuantitatif antar-entitas, yang secara umum diklasifikasikan menjadi hubungan satu-ke-satu (*one-to-one*), satu-ke-banyak (*one-to-many*), serta banyak-ke-banyak (*many-to-many*) [22].

Transformasi pemodelan konseptual ERD ke dalam skema fisik basis data relasional merupakan tahapan kritis yang secara langsung menentukan tingkat integritas dan efisiensi operasi manajemen data. Pemetaan entitas dan kardinalitas relasi yang dianalisis secara akurat akan menghasilkan rancangan tabel-tabel yang ternormalisasi, yang pada gilirannya mampu meminimalisasi redundansi penyimpanan dan mencegah terjadinya inkonsistensi saat proses manipulasi data dilakukan [24]. Dalam arsitektur sistem informasi modern, khususnya aplikasi berbasis web yang harus melayani tingkat konkurensi pertukaran data yang tinggi, desain ERD yang komprehensif menjadi fondasi mutlak untuk memastikan optimalisasi kueri dan skalabilitas pangkalan data. Penerapan metodologi perancangan pangkalan data konseptual menggunakan ERD ini telah terbukti secara empiris mampu memberikan tingkat keandalan yang sangat tinggi dalam mengelola integritas data transaksional, mengamankan rekam jejak relasi entitas, serta mendukung arsitektur pelacakan sumber daya yang presisi pada berbagai sistem manajemen informasi [5].

2.8 UI/UX Design Principles

User Interface (UI) dan *User Experience* (UX) merupakan dua disiplin ilmu yang saling terintegrasi dalam ruang lingkup perancangan antarmuka manusia dan komputer

(*Human-Computer Interaction*). Meskipun dalam praktik pengembangannya sering dianggap sebagai kesatuan, keduanya berfokus pada dimensi perancangan yang berbeda namun saling melengkapi. Menurut Garrett, UI berfokus pada arsitektur visual, elemen interaktif, tata letak, tipografi, dan estetika presentasi dari sebuah sistem perangkat lunak, yang bertujuan untuk mengoptimalkan aksesibilitas fisik serta kejelasan penyampaian informasi [25]. Di sisi lain, UX mencakup ruang lingkup yang lebih holistik dan psikologis, berkaitan erat dengan pengalaman kognitif, respons emosional, serta tingkat kepuasan fungsional pengguna selama berinteraksi dengan sistem tersebut. Desain UX yang berkualitas menuntut pemahaman yang mendalam mengenai perilaku pengguna, memastikan bahwa sistem tidak hanya menyajikan fungsi teknis yang berjalan dengan baik, tetapi juga menghadirkan alur kerja yang intuitif, dapat diprediksi, dan bermakna [26].

Perancangan antarmuka dan pengalaman pengguna yang andal harus dibangun di atas fondasi prinsip-prinsip heuristik usability yang telah teruji secara akademis maupun praktis. Evaluasi usability klasik yang dirumuskan oleh Nielsen menetapkan sejumlah kaidah fundamental, di antaranya meliputi konsistensi desain, visibilitas status sistem, pencegahan kesalahan operasional, serta fleksibilitas kontrol bagi pengguna [27]. Prinsip konsistensi mewajibkan sebuah antarmuka menggunakan konvensi visual, terminologi, dan pola interaksi yang seragam di seluruh modul sistem, sehingga secara drastis meminimalkan kebingungan adaptasi pengguna. Lebih lanjut, perancangan yang berpusat pada pengguna menuntut penyederhanaan antarmuka untuk mengurangi beban kognitif (*cognitive load*), di mana arsitektur informasi harus disajikan secara hierarkis dan logis. Penyediaan umpan balik (*feedback*) yang cepat dan informatif setiap kali pengguna melakukan sebuah tindakan juga menjadi kriteria krusial agar interaksi manusia dengan mesin terasa lebih transparan dan mudah dikendalikan [26].

Transformasi prinsip-prinsip UI/UX ke dalam arsitektur perangkat lunak berbasis web modern merupakan faktor determinan terhadap tingkat adopsi dan efektivitas operasional sebuah sistem informasi. Antarmuka aplikasi web masa kini tidak hanya dituntut untuk memiliki estetika visual yang modern, tetapi juga harus bersifat responsif, mengutamakan kemudahan navigasi, dan terstruktur sedemikian rupa sehingga pengguna dapat menyelesaikan tugas-tugas fungsionalnya dengan tingkat efisiensi yang tinggi. Mengabaikan aspek UX dalam siklus hidup rekayasa perangkat lunak sering kali berakibat fatal pada tingginya rasio kesalahan *input*, frustrasi pengguna akhir, dan penurunan produktivitas akibat alur sistem yang tidak intuitif [25]. Sebaliknya, berbagai penelitian empiris membuktikan bahwa integrasi dan evaluasi kaidah desain UI/UX yang terstruktur sejak fase konseptual secara signifikan mampu meminimalisasi kompleksitas sistem, meningkatkan kepuasan subjektif pengguna, serta memastikan pencapaian tujuan fungsional (*usability success*) pada berbagai platform perangkat lunak yang kompleks [28].

BAB III

METODOLOGI PENELITIAN

3.1 Metode Pengembangan Sistem

Metode pengembangan sistem yang digunakan dalam penelitian ini adalah model Waterfall. Model ini dipilih karena memiliki alur pengembangan yang sistematis, terstruktur, dan berurutan, sehingga sesuai digunakan pada penelitian yang kebutuhan sistemnya telah dapat diidentifikasi sejak awal. Selain itu, model Waterfall memudahkan proses dokumentasi pada setiap tahap pengembangan, mulai dari analisis kebutuhan hingga pemeliharaan sistem [12][11].

Model Waterfall merupakan pendekatan pengembangan perangkat lunak yang membagi proses pengembangan ke dalam beberapa tahap yang dilakukan secara berurutan. Setiap tahap harus diselesaikan terlebih dahulu sebelum melanjutkan ke tahap berikutnya, sehingga hasil dari satu tahap menjadi masukan bagi tahap selanjutnya [29][12]. Dalam penelitian ini, tahapan Waterfall yang digunakan meliputi analisis kebutuhan, perancangan sistem, implementasi, pengujian, dan pemeliharaan.

Tahap analisis kebutuhan dilakukan untuk mengidentifikasi kebutuhan fungsional dan non-fungsional dari sistem yang akan dikembangkan. Pada tahap ini, dilakukan pengumpulan informasi mengenai proses yang berjalan, permasalahan yang dihadapi, serta kebutuhan pengguna terhadap sistem. Hasil dari tahap ini menjadi dasar dalam menyusun rancangan sistem yang akan dibangun [11].

Tahap perancangan sistem dilakukan setelah kebutuhan sistem berhasil diidentifikasi. Pada tahap ini, dilakukan perancangan arsitektur sistem, perancangan basis data, perancangan antarmuka, serta pemodelan sistem menggunakan diagram seperti DFD, ERD, dan UML. Perancangan ini bertujuan untuk memberikan gambaran yang jelas mengenai struktur dan alur kerja sistem sebelum dilakukan proses implementasi [12].

Tahap implementasi merupakan proses penerjemahan rancangan sistem ke dalam bentuk kode program. Pada tahap ini, sistem dikembangkan menggunakan bahasa pemrograman PHP dengan dukungan framework web modern serta basis data MySQL. Implementasi dilakukan sesuai dengan rancangan yang telah dibuat pada tahap sebelumnya agar sistem dapat berjalan sesuai kebutuhan pengguna [11].

Tahap pengujian dilakukan untuk memastikan bahwa sistem yang telah dikembangkan berfungsi sesuai dengan kebutuhan yang telah ditentukan. Pengujian dalam penelitian ini dilakukan menggunakan metode *blackbox testing* untuk memverifikasi setiap fungsi pada sistem, seperti proses input data, pengolahan data, penyimpanan data, dan output

yang dihasilkan. Tahap ini penting untuk memastikan bahwa tidak terdapat kesalahan fungsional pada sistem [12].

Tahap terakhir adalah pemeliharaan. Tahap ini dilakukan setelah sistem digunakan oleh pengguna untuk memperbaiki kesalahan yang mungkin ditemukan, meningkatkan kinerja sistem, atau menyesuaikan sistem dengan kebutuhan baru yang muncul di kemudian hari. Dengan adanya tahap pemeliharaan, sistem dapat terus digunakan secara optimal dan tetap relevan terhadap kebutuhan pengguna [11].

Secara keseluruhan, penerapan model Waterfall dalam penelitian ini memberikan alur pengembangan yang jelas dan terstruktur. Setiap tahap menghasilkan keluaran yang menjadi dasar bagi tahap berikutnya, sehingga proses pengembangan sistem dapat dilakukan secara lebih terkontrol, terdokumentasi dengan baik, dan sesuai dengan tujuan penelitian.

3.2 Tahapan Pengembangan

3.2.1 Planning (Perencanaan)

3.2.2 Analysis (Analisis)

3.2.3 Design (Perancangan)

3.2.4 Implementation (Implementasi)

3.2.5 Testing (Pengujian)

BAB IV

ANALISIS DAN PERANCANGAN SISTEM

4.1 Analisis Sistem

4.1.1 Analisis Sistem Berjalan (Flowmap)

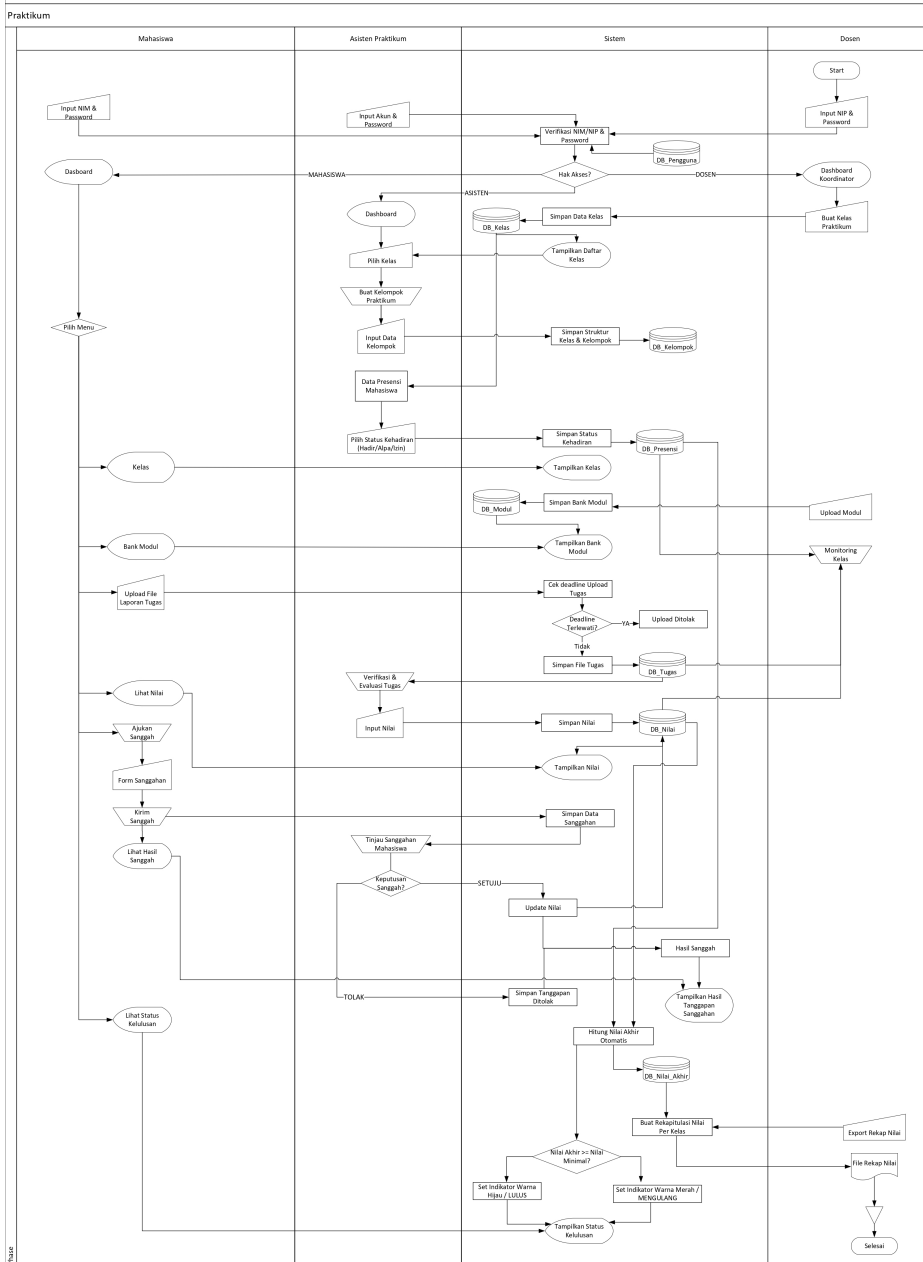
Sistem berjalan merupakan mekanisme operasional yang saat ini diterapkan dalam mengelola kegiatan praktikum sebelum adanya sistem EduLab. Berdasarkan observasi, manajemen praktikum masih dilakukan secara konvensional menggunakan berbagai platform pihak ketiga yang tidak saling terintegrasi.

Berikut adalah uraian proses sistem berjalan:

1. Distribusi materi dan tugas dilakukan asisten melalui grup pesan instan atau folder Google Drive.
2. Mahasiswa mengumpulkan laporan dengan mengunggahnya ke Google Drive atau mengirim langsung ke asisten, yang memicu penumpukan berkas tak terorganisasi dan menyulitkan validasi batas waktu pengumpulan.
3. Asisten mengunduh laporan dan menilai secara manual di Microsoft Excel tanpa rubrik terpadu, yang rentan terhadap inkonsistensi penilaian.
4. Keluhan nilai disampaikan melalui pesan pribadi tanpa dokumentasi resmi, menyulitkan pengawasan oleh dosen.

Untuk mengatasi kendala tersebut, dirancanglah usulan alur kerja baru yang terpusat. Alur sistem usulan ini dapat dilihat secara visual melalui Flowmap pada Gambar 4.1.

Praktikum



4.1.2 Identifikasi Masalah

Berdasarkan hasil analisis terhadap sistem yang berjalan, ditemukan beberapa permasalahan utama yang menghambat efektivitas pelaksanaan praktikum:

- Desentralisasi data di berbagai platform menyebabkan pencarian berkas menjadi sulit dan berisiko kehilangan data.
- Proses pengumpulan manual membuat penegakan batas waktu pengumpulan tidak dapat dilakukan secara otomatis dan akurat.
- Penilaian tanpa rubrik terintegrasi seringkali memunculkan subjektivitas dan perbedaan standar antar asisten praktikum.
- Tidak adanya wadah resmi untuk menangani sanggahan nilai membuat proses penyelesaian keluhan menjadi tidak transparan.

4.1.3 Analisis Kebutuhan Sistem

Tahap analisis kebutuhan sistem bertujuan untuk menjabarkan spesifikasi perangkat lunak yang akan dibangun agar dapat menyelesaikan permasalahan yang telah diidentifikasi. Kebutuhan ini dibagi menjadi dua kategori utama, yaitu kebutuhan fungsional dan kebutuhan non-fungsional.

Kebutuhan Fungsional

Kebutuhan fungsional mendefinisikan fitur-fitur dan proses utama yang harus dapat dilakukan oleh sistem. Untuk sistem EduLab, kebutuhan fungsional meliputi:

- Sistem harus mampu memfasilitasi autentikasi pengguna (dosen, asisten praktikum, mahasiswa) berdasarkan hak akses masing-masing.
- Dosen dapat mengelola data kelas, kelompok praktikum, dan mengunggah modul pembelajaran.
- Sistem harus memungkinkan mahasiswa mengunggah laporan praktikum secara terpusat dengan penegakan otomatis pada batas waktu (deadline).
- Asisten praktikum dapat memberikan nilai langsung melalui sistem menggunakan rubrik penilaian yang telah disiapkan.
- Sistem menyediakan fasilitas bagi mahasiswa untuk mengajukan sanggahan nilai yang nantinya ditinjau oleh asisten praktikum.

Kebutuhan Non-Fungsional

4.2 Perancangan Sistem

4.2.1 Diagram Konteks (DFD Level 0)

4.2.2 DFD Level 1

4.2.3 Perancangan Basis Data (ERD + Relasi Tabel)

4.2.4 Perancangan UML

Use Case Diagram

Activity Diagram

Sequence Diagram

Class Diagram

Collaboration Diagram

State Diagram

4.3 Perancangan Antarmuka (UI/UX)

4.3.1 Wireframe / Mockup

4.3.2 Desain Tampilan

4.3.3 User Flow

BAB V

IMPLEMENTASI DAN PENGUJIAN

5.1 Implementasi

5.1.1 Implementasi Sistem (Frontend & Backend)

5.1.2 Implementasi Database

5.1.3 Screenshot Hasil Sistem

5.2 Pengujian

5.2.1 Metode Pengujian (Black Box)

5.2.2 Hasil Pengujian

5.2.3 Evaluasi Sistem

BAB VI

PENUTUP

6.1 Kesimpulan

6.2 Saran

Daftar Pustaka

- [1] K. C. Laudon and J. P. Laudon, *Management Information Systems: Managing the Digital Firm*, 14th. New Jersey: Pearson, 2016.
- [2] R. Stair and G. Reynolds, *Principles of Information Systems*, 13th. Boston: Cengage Learning, 2018.
- [3] D. T. Bourgeois, *Information Systems for Business and Beyond*. The Saylor Foundation, 2014.
- [4] D. Al-Fraihat, M. Joy, and J. Sinclair, “Evaluating E-learning systems success: An empirical study,” *Computers in Human Behavior*, vol. 102, pp. 67–86, 2020. DOI: 10.1016/j.chb.2019.08.011.
- [5] A. A. Adekunle, B. L. Abolore, G. Mutiu, and A. M. Olalekan, “Design and Implementation of a Web-Based Laboratory Management System for Efficient Resource Tracking,” *Asian Journal of Electrical Sciences*, vol. 13, no. 2, pp. 19–24, 2024. DOI: 10.70112/ajes-2024.13.2.4248.
- [6] P. Ihanola, T. Ahoniemi, V. Karavirta, and O. Seppälä, “Review of recent systems for automatic assessment of programming assignments,” in *Proceedings of the 10th Koli Calling International Conference on Computing Education Research*, New York, NY, USA: Association for Computing Machinery, 2010, pp. 86–93. DOI: 10.1145/1930464.1930480.
- [7] P. Dawson, “Assessment rubrics: towards a clearer and more replicable design, research and practice,” *Assessment & Evaluation in Higher Education*, vol. 42, no. 3, pp. 347–360, 2017. DOI: 10.1080/02602938.2015.1111294.
- [8] D. J. Nicol and D. Macfarlane-Dick, “Formative assessment and self-regulated learning: A model and seven principles of good feedback practice,” *Studies in Higher Education*, vol. 31, no. 2, pp. 199–218, 2006. DOI: 10.1080/03075070600572090.
- [9] C. Romero and S. Ventura, “Educational data mining and learning analytics: An updated survey,” *WIREs Data Mining and Knowledge Discovery*, vol. 10, no. 3, e1355, 2020. DOI: 10.1002/widm.1355.
- [10] V. M. Bradley, “Learning Management System (LMS) use with online instruction,” *International Journal of Technology in Education*, vol. 4, no. 1, pp. 68–92, 2021. DOI: 10.46328/ijte.36.

- [11] I. Sommerville, *Software Engineering*, 10th. London: Pearson, 2016.
- [12] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th. New York: McGraw-Hill Education, 2014.
- [13] J. Alqurni, "Assessing the Usability of E-Learning Software Among University Students: A Study on Student Satisfaction and Performance," *International Journal of Information Technology and Web Engineering*, vol. 18, no. 1, pp. 1–26, 2023. DOI: 10.4018/IJITWE.329198.
- [14] S. Sarwosri, A. Muklason, and W. Astrini, "Software Quality Measurement for Functional Suitability, Performance Efficiency, and Reliability Characteristics Using Analytical Hierarchy Process," *JOIV : International Journal on Informatics Visualization*, vol. 7, no. 4, pp. 1080–1086, 2023. DOI: 10.62527/joiv.7.4.2441.
- [15] T. Thesing, C. Feldmann, and M. Burchardt, "Agile versus Waterfall Project Management: Decision Model for Selecting the Appropriate Approach to a Project," *Procedia Computer Science*, vol. 181, pp. 746–756, 2021. DOI: 10.1016/j.procs.2021.01.227.
- [16] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*, 2nd. Boston, MA: Addison-Wesley Professional, 2005.
- [17] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd. Boston, MA: Addison-Wesley Professional, 2004.
- [18] J. Conallen, "Modeling Web Application Architectures with UML," *Communications of the ACM*, vol. 42, no. 10, pp. 63–70, 1999. DOI: 10.1145/317208.317220.
- [19] A. Dennis, H. Wixom, and D. Tegarden, *Systems Analysis and Design: An Object-Oriented Approach with UML*, 5th. Hoboken, NJ: John Wiley & Sons, 2015.
- [20] J. Valdez, A. Zavala, J. Cabrera, et al., "Educational Software Design Using UML and Open Source Tools," in *6th International Conference in Software Engineering Research and Innovation (CONISOFT)*, IEEE, 2018, pp. 56–61. DOI: 10.1109/CONISOFT.2018.8645851.
- [21] K. E. Kendall and J. E. Kendall, *Systems Analysis and Design*, 9th. Upper Saddle River, NJ: Pearson, 2014.
- [22] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th. Hoboken, NJ: Pearson, 2015.
- [23] P. P.-S. Chen, "The Entity-Relationship Model: Toward a Unified View of Data," *ACM Transactions on Database Systems (TODS)*, vol. 1, no. 1, pp. 9–36, 1976. DOI: 10.1145/320434.320440.

- [24] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th. New York, NY: McGraw-Hill Education, 2019.
- [25] J. J. Garrett, *The Elements of User Experience: User-Centered Design for the Web and Beyond*, 2nd. Berkeley, CA: New Riders, 2010.
- [26] D. Norman, *The Design of Everyday Things: Revised and Expanded Edition*. New York, NY: Basic Books, 2013.
- [27] J. Nielsen, *Usability Engineering*. San Francisco, CA: Morgan Kaufmann, 1994.
- [28] C. Lallemand, G. Gronier, and V. Koenig, “User experience: A concept without consensus? Exploring practitioners’ perspectives through an international survey,” *Computers in Human Behavior*, vol. 43, pp. 35–48, 2015. DOI: 10.1016/j.chb.2014.10.048.
- [29] W. W. Royce, “Managing the Development of Large Software Systems: Concepts and Techniques,” *Proceedings of the 9th IEEE-WESTCON*, pp. 1–9, Aug. 1970.